

Practical No.:	08
Practical Title :	To understand Jenkins Master-Slave Architecture and scale your Jenkins standalone implementation by implementing slave nodes
Aim:	To understand Jenkins Master-Slave architecture and scale the system by configuring and managing slave nodes.
Objective:	To understand the Jenkins Master-Slave architecture and its components. To install and configure Jenkins Master on a server. To set up and configure one or more Jenkins Slave nodes.

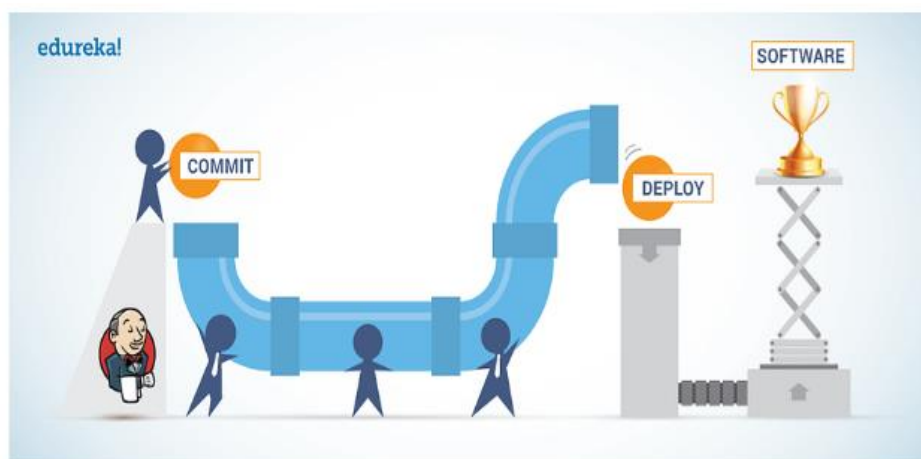
Theory:**What is Jenkins?**

Jenkins is one of the most important tools in DevOps. Jenkins is used in the Continuous Integration stage of DevOps. In this article, I am going to talk about the Jenkins Master and Slave architecture.

Jenkins is an open-source automation tool written in Java with plugins built for Continuous Integration purposes. Jenkins is used to building and test your software projects continuously making it easier for developers to integrate changes to the project, and making it easier for users to obtain a fresh build. It also allows you to continuously deliver your software by integrating with a large number of testing and deployment technologies.

With Jenkins, organizations can accelerate the software development process through automation. Jenkins integrates development life-cycle processes of all kinds, including build, document, test, package, stage, deploy, static analysis, and much more.

Jenkins achieves Continuous Integration with the help of plugins. Plugins allow the integration of Various DevOps stages. If you want to integrate a particular tool, you need to install the plugins for that tool. For example Git, Maven 2 project, Amazon EC2, HTML publisher, etc.

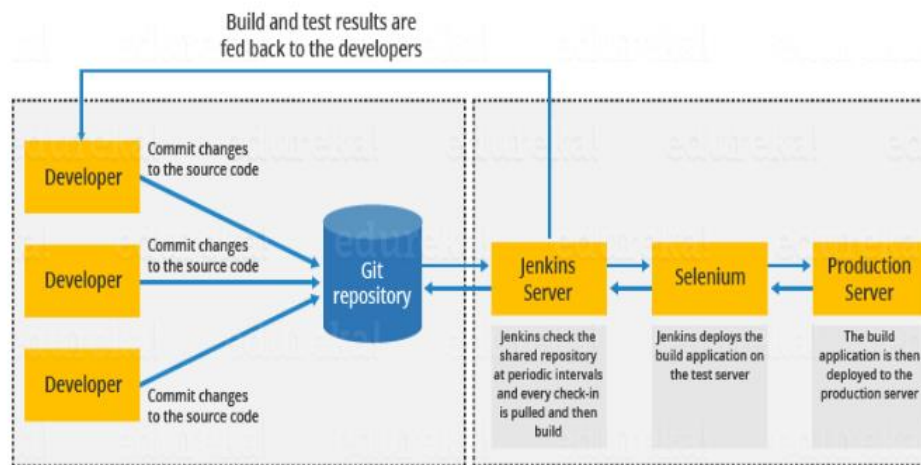
**Advantages of Jenkins :**

- It is an open-source tool with great community support.
- Too easy to install.
- It has 1000+ plugins to ease your work. If a plugin does not exist, you can code it and share it with the community.
- It is free of cost.

- It is built with Java and hence, it is portable to all the major platforms.

Jenkins Architecture :

Let us have a look at the Jenkins Architecture below diagram depicts the same.



This single Jenkins server was not enough to meet certain requirements like:

- Sometimes you might need several different environments to test your builds. This cannot be done by a single Jenkins server.
- If larger and heavier projects get built on a regular basis then a single Jenkins server cannot simply handle the entire load.

To address the above-stated needs, Jenkins distributed architecture came into the picture

Jenkins Distributed Architecture:

Jenkins uses a Master-Slave architecture to manage distributed builds. In this architecture, Master and Slave communicate through TCP/IP protocol.

Jenkins Master:

Your main Jenkins server is the Master. The Master's job is to handle:

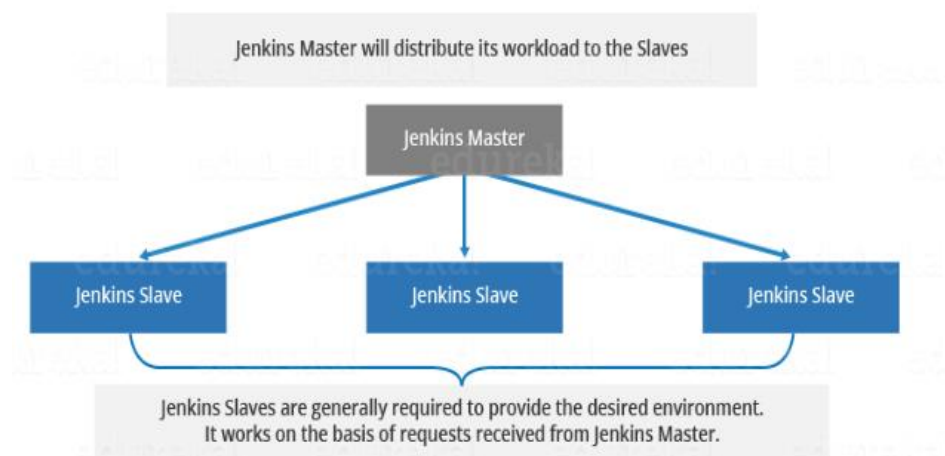
- Scheduling build jobs.
- Dispatching builds to the slaves for the actual execution.
- Monitor the slaves (possibly taking them online and offline as required).
- Recording and presenting the build results.
- A Master instance of Jenkins can also execute build jobs directly.

Jenkins Slave:

A Slave is a Java executable that runs on a remote machine. Following are the characteristics of Jenkins Slaves:

- It hears requests from the Jenkins Master instance.
- Slaves can run on a variety of operating systems.
- The job of a Slave is to do as they are told to, which involves executing build jobs dispatched by the Master.
- You can configure a project to always run on a particular Slave machine or a particular type of Slave machine, or simply let Jenkins pick the next available Slave.

The diagram below is self-explanatory. It consists of a Jenkins Master which is managing three Jenkins Slave.

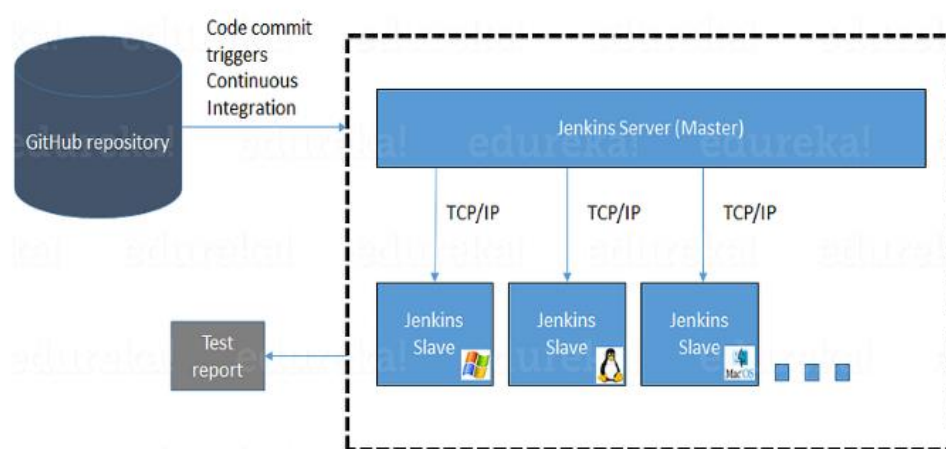


How Jenkins Master and Slave Architecture works?

Now let us look at an example in which we use Jenkins for testing in different environments like

Ubuntu, MAC, Windows, etc.

The diagram below represents the same:



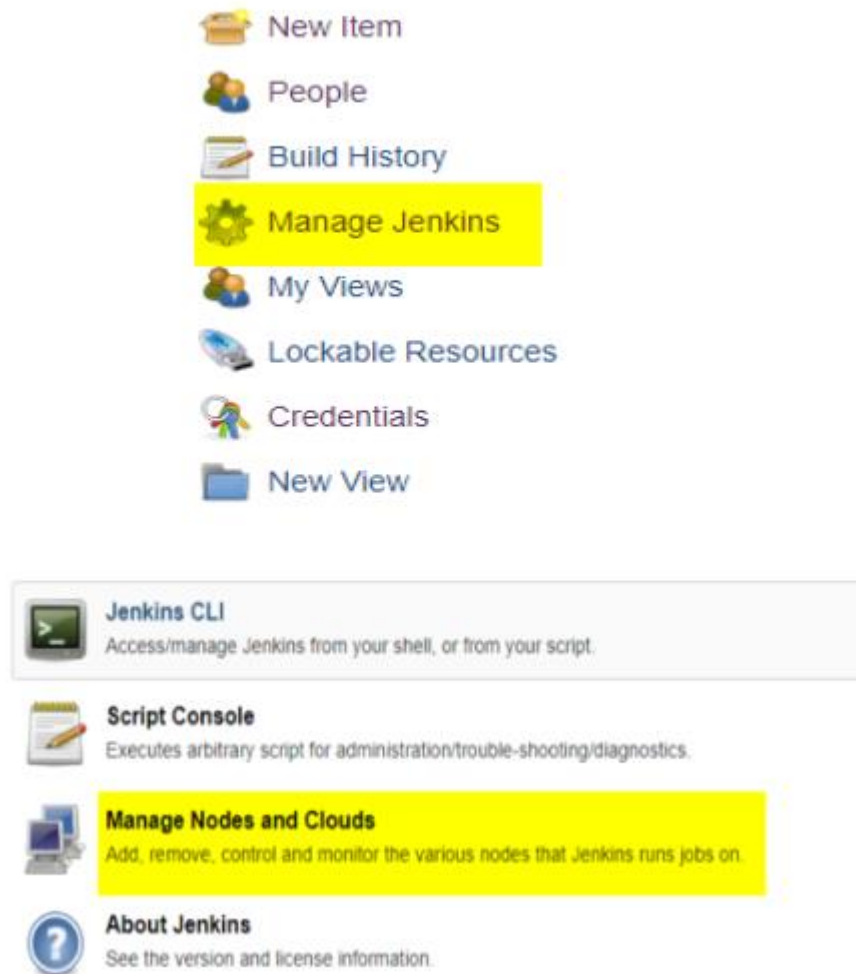
4

The above image represents the following functions:

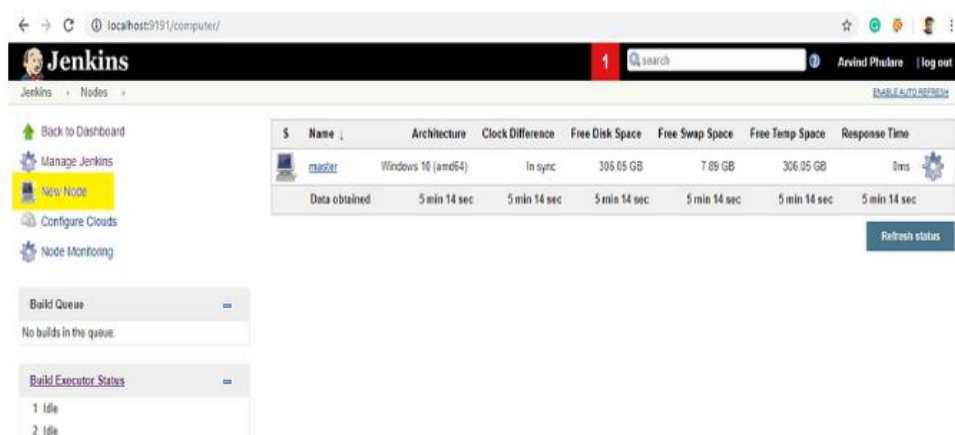
- Jenkins checks the Git repository at periodic intervals for any changes made in the source code.
- Each builds requires a different testing environment which is not possible for a single Jenkins server. In order to perform testing in different environments, Jenkins uses various Slaves as shown in the diagram.
- Jenkins Master requests these Slaves to perform testing and to generate test reports.

How to setup Jenkins Master and Slaves?

1. Go to the Manage Jenkins section and scroll down to the section of Manage Nodes



2. Click on New Node



3. Give a name for the node, choose the Permanent Agent option, and click on Ok

Jenkins - Nodes

Back to Dashboard
Manage Jenkins
New Node
Configure Clouds
Node Monitoring

Build Queue
No builds in the queue.

Build Executor Status
1 idle
2 idle

Node name: Slave1

☒ Permanent Agent
Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

OK

4. Enter the details of the node slave machine.

Here no. of executors in nothing but no. of jobs that this slave can run parallelly. Here we have kept it to 2. The Labels for which the name is entered as “Slave1” is what can be used to configure jobs to use this slave machine. Select Usage to Use this node as much as possible. For the launch method, we select the option of “Launch agent by connecting it to the master”. If this option is not visible then go to Jenkins home page -> Manage Jenkins -> Configure Global Security. Here in the Agents section click on Random and Save it. Now you will find the required option. Enter Custom WorkDir path as the workspace of your slave node. In Availability select “Keep this agent online as much as possible”. Click on Save

Configure
Build History
Load Statistics
Script Console
Log
System Information
Disconnect

Build Executor Status
1 idle
2 idle

of executors: 2

Remote root directory: C:/Users/Arvind Phulare/Desktop/slave_vs

Labels: slave1

Usage: Use this node as much as possible

Launch method: Launch agent by connecting it to the master

☐ Disable WorkDir

Custom WorkDir path: C:/Users/Arvind Phulare/Desktop/slave_vs

Internal data directory: remoteing

☐ Fail if workspace is missing

Advanced...

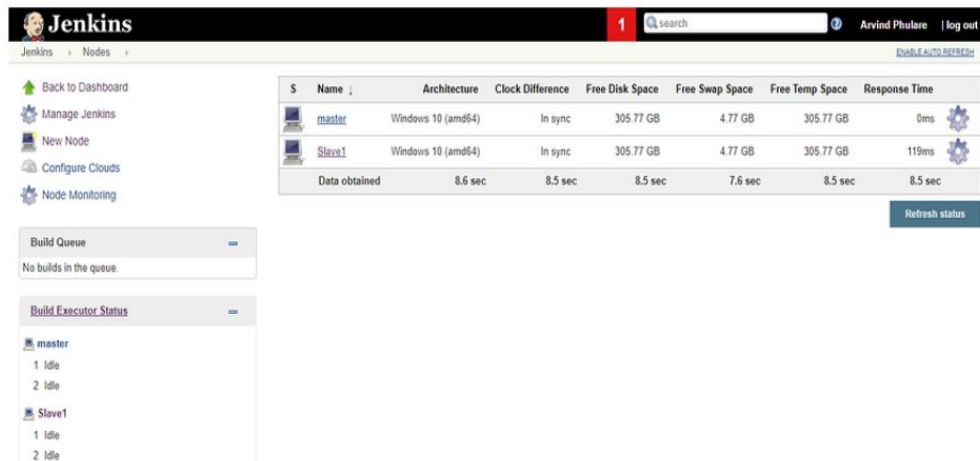
Availability: Keep this agent online as much as possible

Node Properties

☐ Disable deferred wipeout on this node

Save

Once you complete the above steps, the new node machine will initially be in an offline state but will come online if all the settings in the previous screen were entered correctly. One can at any time make the node slave machine offline if required



The Jenkins Nodes page displays a table of nodes and their status. The table has columns for Name, Architecture, Clock Difference, Free Disk Space, Free Swap Space, Free Temp Space, and Response Time. The nodes listed are master and Slave1, both running Windows 10 (amd64) and in sync. The table also shows data obtained for each node.

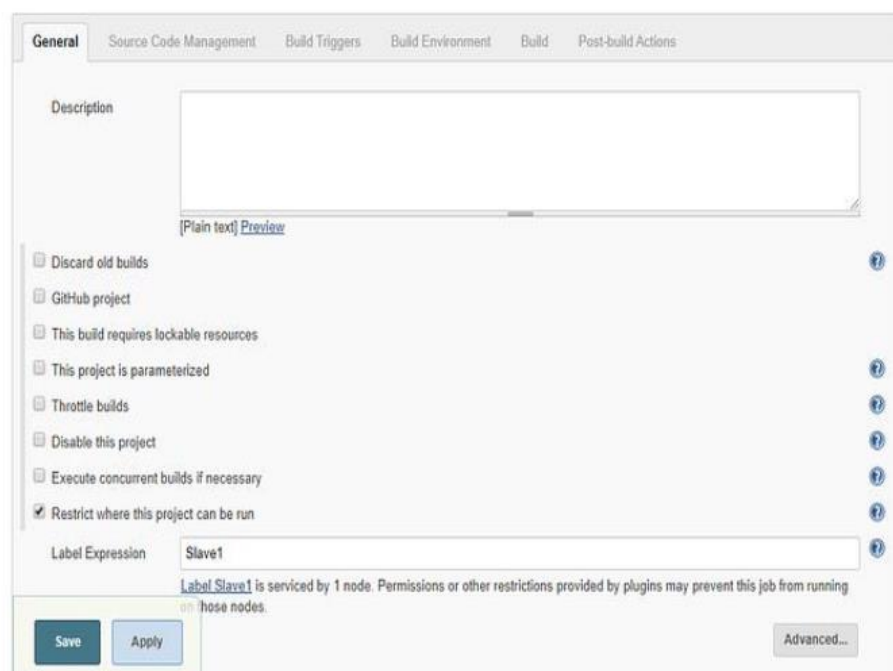
S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	master	Windows 10 (amd64)	In sync	305.77 GB	4.77 GB	305.77 GB	0ms
	Slave1	Windows 10 (amd64)	In sync	305.77 GB	4.77 GB	305.77 GB	119ms
	Data obtained	8.6 sec	8.5 sec	8.5 sec	7.6 sec	8.5 sec	8.5 sec

Buttons: Back to Dashboard, Manage Jenkins, New Node, Configure Clouds, Node Monitoring, Build Queue, Build Executor Status, Refresh status.

5. Now since your slave is up and running, let's execute a job on the slave.

For that, I already have an existing job and I will run this job on this slave. Open this job and click on configure. Now here in the General section, click on “Restrict where this project can be run”.

Here in Label Expression, enter the name of the slave and save it. Now click on Build now and see the output of this job. Everything is correct you will see the output as Success.



The Jenkins Job Configuration page shows the General tab. The Description field is empty. The Restrict where this project can be run checkbox is checked. The Label Expression field contains 'Slave1'. The text below the field states: 'Label Slave1 is serviced by 1 node. Permissions or other restrictions provided by plugins may prevent this job from running on those nodes.'

Buttons: Save, Apply, Advanced...

Conclusion: